BioSpecInfo

Guida al deploy — BioSpecInfo

Samuele Pio Provenzano

Versione **bsi-v220** · commit `784d87b`

Generato il 2026-10-16

Documento generato automaticamente dal sorgente Markdown versionato nel repository. La versione e il commit stampati in ogni piè di pagina identificano esattamente lo stato del software descritto.

Guida al deploy — BioSpecInfo

Campo	Valore
Software	BioSpecInfo
Versione descritta	`bsi-v220`
Scopo	Procedura operativa per pubblicare, aggiornare e ripristinare l'applicazione.

0. Cosa si pubblica, in pratica

BioSpecInfo è un insieme di **file statici**. Non c'è nulla da compilare, nessun processo da tenere acceso, nessun database da provvisionare.

Build	Nessuna. I file del repository *sono* l'applicazione.
Runtime server	Nessuno
Variabili d'ambiente	Nessuna per l'applicazione. Solo il proxy opzionale ne ha (§4).
Database	Nessuno lato server
Requisito del server	Servire file statici su HTTPS con i MIME corretti

L'unico requisito non ovvio: il server deve restituire `application/wasm` per i file `.wasm`, altrimenti RDKit e SQLite non si avviano. GitHub Pages, Netlify, Vercel e Cloudflare Pages lo fanno già.

1. Ambienti

Ambiente	Come si ottiene	A cosa serve
Sviluppo	`python3 -m http.server 8899` dalla radice	Lavoro quotidiano e banchi di prova
Anteprima	Un ramo pubblicato su Pages, o `npx serve`	Verifica prima dell'unione
Produzione	GitHub Pages dal ramo `main`	`samupropio1-ship-it.github.io/BioSpecInfo-v11/`

> **Perché serve un server anche in sviluppo.** Aprire `index.html` con un doppio clic (`file://`) non funziona: RDKit e SQLite sono moduli WebAssembly e richiedono un contesto HTTP. È il motivo per cui anche i banchi di prova girano contro un server locale.

2. Pubblicazione in produzione

2.1 Procedura

```
# 1. dal ramo di lavoro, batteria completa
python3 -m http.server 8899 &
node tools/genera-evidenza.js          # deve chiudersi con 0 falliti

# 2. incrementare la versione — con lo strumento, MAI a mano
node tools/porta-versione.js bsi-v<NNN+1> --prova   # prima si guarda
node tools/porta-versione.js bsi-v<NNN+1>           # poi si scrive

# 3. rigenerare SBOM, evidenza, PDF e pacchetti — IN QUEST'ORDINE
#    I pacchetti imbustano i PDF e l'evidenza: se si costruiscono prima,
#    portano l'impaginato e il rapporto della versione PRECEDENTE.
#    genera-pacchetti adesso se ne accorge e si ferma, ma meglio non arrivarci.
node tools/genera-sbom.js
node tools/genera-evidenza.js       # la batteria; serve un server su :8899
node tools/genera-pdf.js            # 36 PDF; ogni piè di pagina porta versione e commit
node tools/genera-pacchetti.js      # i tre pacchetti; fallisce su link rotti, PDF o evidenza
vecchi

# 4. unire in main
git add -A && git commit -m "…"
git push -u origin <ramo>
#    poi pull request e squash merge
```

GitHub Pages pubblica automaticamente da `main`: il workflow *pages build and deployment* impiega circa un minuto.

2.1-bis Pubblicare la release della versione

L'applicazione è già online appena `main` si aggiorna; la **release** serve a un'altra cosa: dare a chi valuta un punto fermo da scaricare — un tag, le note di versione e i pacchetti di consegna allegati.

Si pubblica dal repository, non da fuori:

> **Actions → release → Run workflow**, campo `versione`: `bsi-v<NNN>`.

Il workflow crea il tag sul commit corrente, apre la release e le allega i tre pacchetti di `consegna/`. L'archivio completo — `models/` incluse, circa 124 MB — lo genera GitHub da sé come *Source code (zip)*.

Perché dal repository e non da riga di comando. Le credenziali con cui una sessione remota lavora al repository sono autorizzate a scrivere sui rami e **non sui tag**: la `POST` a `git-receive-pack` per `refs/tags/…` risponde `403 Forbidden`, mentre un ramo nuovo passa. Il token di Actions, invece, scrive sul repository che lo ospita.

Quello che il workflow **si rifiuta** di fare:

Condizione	Esito
nel `CHANGELOG.md` non c'è una voce `## [bsi-v<NNN>]` con contenuto	fallisce: una release senza note sembra una versione e non dice che cosa cambia
in `consegna/` ci sono meno di tre pacchetti	fallisce: un allegato mancante renderebbe la release incompleta in silenzio
il tag non punta al commit da cui vengono i pacchetti	fallisce: una release deve avere una sola provenienza

L'ultima riga esiste per un difetto reale. Rieseguendo il workflow per aggiornare i pacchetti, `gh release edit` aggiornava note e allegati ma **non spostava il tag**: l'archivio *Source code (zip)*, che GitHub genera dal tag, restava al commit della prima pubblicazione mentre i pacchetti venivano da uno successivo. Due provenienze, e niente che lo dicesse. Ora il tag viene spostato, e un passo successivo non lo dà per buono: richiede a GitHub dove punta e confronta.

Il workflow è **rieseguibile**: se la release esiste già, aggiorna note, allegati e tag invece di fallire a metà. Spostare un tag già pubblicato non è gratis — chi l'ha già scaricato ha un contenuto diverso — quindi vale per una release appena pubblicata, non per una vecchia: per quella si pubblica una versione nuova.

2.1-ter Perché il cambio di versione non si fa con una sostituzione

Sembra l'operazione più innocua del mondo: si cerca `bsi-v<NNN>` e si scrive `bsi-v<NNN+1>`. **Per tre rilasci di fila è stato fatto così, e ogni volta ha riscritto delle frasi storiche** — frasi che raccontano un fatto accaduto a una versione passata. Quello che ne è uscito stava dentro una dichiarazione di conformità firmata:

nel documento c'era scritto	la verità
la password era stata **cambiata alla versione corrente**	era stata cambiata alla versione `bsi-v188`
il tag era stato **spostato al commit della versione corrente**	l'incidente era accaduto alla versione `bsi-v188`
le trentasei voci erano state **aggiunte nella versione corrente**	erano state aggiunte in `bsi-v188`
«incrementa `CACHE` (es. `bsi-vN` → `bsi-vN`)»	un esempio uguale a se stesso

Nessun controllo poteva accorgersene: ogni riga citava una versione che **esiste**, nel documento che descrive quella versione.

> Questa tabella è scritta con dei segnaposto e non con i numeri veri perché i due controlli di cui si parla qui sotto **la boccerebbero** — ed è la dimostrazione che funzionano.

`tools/porta-versione.js` tocca **solo** le righe che DICHIARANO la versione — intestazioni, piè di pagina, distintivo del README, nomi dei pacchetti, le due righe di codice — e tutto il resto lo **elenca** perché lo si legga. Con `--prova` non scrive niente.

E due controlli in `tools/verifica-documenti.js` chiudono la porta: nessuna frase al passato può nominare la versione corrente, e nessun esempio di aggiornamento può mostrare la stessa versione da entrambe le parti.

> **Scrivere nominando la cosa, non il numero.** «Dalle tabelle di Pretsch» invece di «dalla versione tale»: si legge meglio e non invecchia.

2.2 Le due righe della versione

`CACHE` in `sw.js` e `BSI_APP_VERSION` in `index.html` **devono coincidere**.

- `CACHE` determina se il Service Worker considera la cache obsoleta: senza

incremento, i dispositivi continuano a servire i file vecchi e la pubblicazione non arriva a nessuno.

- `BSI_APP_VERSION` è ciò che l'utente vede nella finestra «Aggiornamenti» e ciò

che il controllo confronta con il `sw.js` su GitHub.

Sono disallineate una volta in passato — la seconda era ferma da mesi — e il risultato è stato che il controllo aggiornamenti confrontava un numero sbagliato.

2.3 Come arriva l'aggiornamento agli utenti

1. Il browser rileva un `sw.js` diverso e installa il nuovo Service Worker.
2. `skipWaiting()` lo attiva subito; le cache vecchie vengono eliminate.
3. Il Service Worker **avvisa** le schede aperte (`BSI_SW_UPDATED`) — non le

ricarica d'ufficio.

1. La pagina decide: se non c'è lavoro in corso si ricarica da sola; se

l'assistente sta rispondendo o c'è testo non salvato, mostra «Aggiorna ora / Più tardi».

Chi apre l'app in quel momento riceve già la versione nuova.

---

3. Alternative di pubblicazione

L'applicazione non dipende da GitHub Pages. Qualunque host di file statici va bene, purché su HTTPS (il Service Worker lo richiede).

Piattaforma	Comando / configurazione	Note
GitHub Pages	Settings → Pages → branch `main` , cartella `/`	In uso. Gratuito, dominio personalizzabile
Netlify	`netlify deploy --prod --dir=.`	Nessun comando di build
Vercel	`vercel --prod`	Progetto statico, nessun framework
Cloudflare Pages	`npx wrangler pages deploy .`	Comodo se si usa già il proxy
VPS con nginx	Copia dei file in `/var/www/biospecinfo`	Vedi configurazione sotto

Configurazione nginx

```
server {
    listen 443 ssl http2;
    server_name biospecinfo.example.org;
    root /var/www/biospecinfo;

    # Senza questo tipo MIME, RDKit e SQLite non si avviano
    types { application/wasm wasm; }

    # sw.js non va messo in cache dal CDN: è ciò che annuncia le versioni nuove
    location = /sw.js {
        add_header Cache-Control "no-cache, no-store, must-revalidate";
    }

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

Regola valida ovunque: `sw.js` non va messo in cache

Se una CDN conserva `sw.js`, il browser continua a vedere la versione vecchia e **nessun aggiornamento arriva mai**, per quanto si pubblichi. Vale per ogni piattaforma.

---

4. Proxy opzionale — deploy separato

Il proxy è un componente **distinto**, di proprietà dell'utente, pubblicato sul suo account Cloudflare. Non è necessario per usare l'applicazione.

```
cd proxy
npx wrangler login
npx wrangler deploy
# l'indirizzo viene stampato: https://spectra-proxy.<account>.workers.dev

npx wrangler secret put GROQ_KEYS      # una o più chiavi, separate da virgola
npx wrangler secret put GEMINI_KEYS    # facoltativo
```

Poi si incolla l'indirizzo nel riquadro «Proxy» dell'assistente, dentro l'app. Gli stessi quattro passi sono disponibili **nell'applicazione**, con i comandi pronti da copiare.

Configurazione consigliata

```
# Senza questa variabile il proxy accetta richieste da qualunque origine
npx wrangler secret put ORIGINI
# valore: https://samupropio1-ship-it.github.io
```

Variabile	Default	Perché toccarla
`ORIGINI`	*(vuoto = tutte)*	**Impostarla sempre.** Un proxy aperto è utilizzabile da chiunque, a spese della tua quota.
`LIMITE_IP`	`20`	Richieste per IP nella finestra
`TETTO_GIORNO`	`2000`	Tetto complessivo giornaliero

> **Le chiavi non vanno mai in `wrangler.toml`.** Quel file è versionato: una chiave scritta lì finisce su GitHub e i bot che lo scandagliano la trovano in poche ore. Si usa `wrangler secret put`, che le tiene fuori dal repository.

---

5. Dominio e certificato

Aspetto	Situazione
HTTPS	Obbligatorio: senza, il Service Worker non si registra e l'app non funziona offline
Certificato	Fornito e rinnovato automaticamente da GitHub Pages
Dominio personalizzato	Settings → Pages → Custom domain, più un record `CNAME` verso `<utente>.github.io`

---

6. Monitoraggio

Non esiste un server da monitorare: non ci sono processi che possano cadere, né risorse che si esauriscano.

Cosa osservare	Come
Esito della pubblicazione	GitHub → Actions → *pages build and deployment*
Disponibilità	Un controllo HTTP su `index.html` (UptimeRobot o simili)
Errori lato client	Console del browser. La telemetria remota esiste ma è **disattivata** (`BSI_TELEMETRY_URL` vuoto): attivarla richiede una decisione consapevole sul trattamento dei dati.
Consumo del proxy	Cloudflare → Workers → Analytics

---

7. Ripristino e procedure d'emergenza

7.1 Tornare alla versione precedente

```
git revert <commit>
git push origin main
```

Pages ripubblica in circa un minuto. I dispositivi ricevono la versione ripristinata come un normale aggiornamento — **purché il `CACHE` cambi**: un revert che riporta anche il numero di versione all'indietro va bene, perché conta che il valore sia *diverso*, non che sia maggiore.

7.2 «Ho pubblicato ma non vedo le modifiche»

Nell'ordine:

1. **La versione è stata incrementata?** È la causa nel novanta per cento dei

casi. Senza, il Service Worker serve la cache.

1. **Pages ha concluso?** GitHub → Actions.
2. **Il `sw.js` è in cache da una CDN?** Deve rispondere `no-cache`.
3. Dall'app: menu ✨ → **Aggiornamenti** → *Controlla*, che forza un

`registration.update()`.

1. In ultima istanza: strumenti per sviluppatori → Application → Service Workers

→ *Unregister*, poi ricaricare.

7.3 «L'assistente non risponde più»

Non è un problema di pubblicazione: riguarda il fornitore AI. Nell'app, il pulsante 🔌 **Prova** misura in pochi secondi quali fornitori il dispositivo riesce davvero a contattare, senza bisogno di alcuna chiave.

7.4 Ripristino dei dati dell'utente

I dati risiedono sul dispositivo e **non esistono copie sul server**: non è possibile ripristinarli da remoto. È una conseguenza diretta dell'architettura local-first, ed è dichiarata all'utente. Le esportazioni (note, documenti) restano la sola forma di copia di sicurezza.

---

8. Lista di controllo prima di pubblicare

- ☐ `node tools/genera-evidenza.js` si chiude con **0 falliti**
- ☐ `CACHE` in `sw.js` e `BSI_APP_VERSION` in `index.html` incrementate e **uguali**
- ☐ `node tools/genera-sbom.js` rieseguito
- ☐ `node tools/genera-pdf.js` rieseguito — altrimenti gli allegati descrivono la versione precedente
- ☐ `node tools/genera-pacchetti.js` rieseguito e chiuso con **0 collegamenti rotti**
- ☐ Rapporto di evidenza rigenerato sul commit che si pubblica
- ☐ Eventuali difformità nuove annotate in `09-Release-Conformance-Statement.md` §4
- ☐ Nessun marcatore di conflitto (`verifica_guida` lo controlla su 58 file)
- ☐ Nessuna chiave API nei file versionati
- ☐ Il debito di accessibilità **non è cresciuto** — se è sceso, rigenerare il

riferimento con `node tools/verifica-accessibilita.js --aggiorna-riferimento` e registrarlo nel commit

- ☐ La copertura di codice **non è scesa** — `audit_copertura` fallisce se lo è

Documento aggiornato alla versione `bsi-v220`.